

Звіт до лабораторної роботи № 1

Міністерство освіти і науки України
Національний університет «Львівська політехніка»
Інститут ІКТА
Кафедра ЗІ

Тема: Основи розробки на Python, Git та стандарти стилю коду

Дисципліна: Програмування скриптовими мовами

Студент: Плохута Назар

Група: КБ-205

Лектор: Кольченко В.В.

Керівник лабораторної роботи: Синявський О.Ю.

Дата: 01 жовтня 2026 р.

Львів, 2026 р.

Варіант: 9.

Мета

1. Навчитися працювати з Git, GitHub і віртуальним середовищем.
2. Розкласти код проєкту по модулях, щоб його можна було використати повторно.
3. Оформити код за PEP-8 і перевірити його аналізатором Ruff.
4. Виконати три завдання варіанта: оцінку паролів, контроль доступу, хешування з записом у CSV і журнал у JSON.

Структура проєкту

```
cybersecurity-python-labs/
├── .gitignore
├── requirements.txt
├── ruff.toml
├── shared/
│   ├── __init__.py
│   └── student.py
└── labs/
    └── lab01/
        ├── __init__.py
        ├── main.py
        ├── task1.py
        ├── task2.py
        ├── task3.py
        └── data/
            ├── users.csv
            └── log.json
```

У `shared/student.py` записані ПІБ, група і номер варіанта. `main.py` імпортує їх і по черзі запускає три завдання.

Віртуальне середовище створене командою `python -m venv .venv`. Каталог `.venv` доданий у `.gitignore`. Туди само додані `__pycache__`, кеш Ruff, `users.csv` і `log.json`. Залежність проєкту: `ruff==0.16.9`.

Запуск з кореня репозиторію:

```
source .venv/bin/activate
python labs/lab01/main.py
```

Завдання 1. Аналізатор паролів

Вхідні дані варіанта 9:

Пароль	Довжина
--------	---------

Digital@F0r3nsics	17
plain	5
Encrypt10n@Key	14
member	6
Security@Audit2023	18
regular	7
Hack3r@D3fense	14
ordinary	8
Threat@Intel	12
usual	5

Критерії: мінімальна довжина 8, обов'язкові цифра, велика літера і спеціальний символ.

Заборонені паролі: plain, member, regular, ordinary, usual, user. Слово user є в множині заборони, але його немає в початковому списку.

Перед оцінкою програма через random.sample бере три різні індекси і дописує відповідні паролі в кінець списку. Рівень рахується вже по розширеному списку.

Порядок перевірки у classify_password:

1. **Заборонений** — пароль є в множині заборони або коротший за 8 символів.
2. **Дуже сильний** — є цифра, велика літера і спецсимвол, довжина не менша за 12 і пароль зустрічається в списку один раз.
3. **Сильний** — є всі три групи символів, а довжина менша за 12.
4. **Середній** — довжина не менша за 8 і виконана лише частина груп.
5. **Слабкий** — пароль не заборонений і не потрапив у вищі рівні.

Для початкового списку, до додавання копій, рівні такі:

Пароль	Рівень	Чому
Digital@F0r3nsics	Дуже сильний	усі групи, довжина 17, один екземпляр
plain	Заборонений	є в множині заборони, довжина 5
Encrypt10n@Key	Дуже сильний	усі групи, довжина 14, один екземпляр
member	Заборонений	є в множині заборони, довжина 6
Security@Audit2023	Дуже сильний	усі групи, довжина 18, один екземпляр
regular	Заборонений	є в множині заборони, довжина 7
Hack3r@D3fense	Дуже сильний	усі групи, довжина 14, один екземпляр
ordinary	Заборонений	є в множині заборони; довжина 8, поріг довжини пройдено
Threat@Intel	Середній	довжина 12, є велика літера і спецсимвол, цифри немає
usual	Заборонений	є в множині заборони, довжина 5

Пароля з усіма групами символів і довжиною від 8 до 11 у варіанті немає, тому рівень «Сильний» на цих десяти рядках не з'являється. Якщо скопійований пароль до того був дуже сильним, обидва входування перестають бути унікальними і отримують рівень «Слабкий»: довжина вже не менша за 12, тож під визначення «Сильний» вони не підходять.

Результат виводиться таблицею: номер, пароль, рівень. Перед таблицею друкуються вибрані індекси.

Завдання 2. Контроль доступу

Користувачі варіанта:

Логін	Роль	Допуск	Підрозділ	Активний
cloud_architect	cloud_security	4	Cloud	так
devops_engineer	devops	3	DevOps	так
qa_tester	quality_assurance	2	QA	так
partner_access	partner	2	Partnership	так
migrated_user	migrated	1	Migration	ні

Заблоковані: migrated_user, container_breach, pipeline_compromise.

Ресурси і назви рівнів. Число замінюється назвою з кортежу: 1 — Development, 2 — Staging, 3 — Production, 4 — Critical Infrastructure. На екран виводиться лише назва.

Ресурс	Рівень	Назва
cloud_configs	4	Critical Infrastructure
deployment_pipelines	3	Production
test_environments	2	Staging
partner_apis	2	Staging
infrastructure_code	4	Critical Infrastructure
shared_resources	1	Development
container_registry	3	Production
secrets_vault	4	Critical Infrastructure
build_artifacts	2	Staging
public_endpoints	1	Development

Перевірка кожної пари йде в такому порядку:

1. Логіна немає в словнику `users` — `DENY (User not found)`.
2. Логін є в `blocked_users` — `DENY (User is blocked)`.
3. Поле `active` дорівнює `False` — `DENY (Account inactive)`.
4. Допуск не менший за рівень ресурсу — `ALLOW`.
5. Інакше — `DENY (Insufficient clearance)`.

Рядок виводу має вигляд `user=... resource=... -> ALLOW` або `user=... resource=... -> DENY (причина)`.

Що виходить на даних варіанта:

- `cloud_architect` (допуск 4) отримує `ALLOW` до всіх десяти ресурсів.
- `devops_engineer` (допуск 3) отримує `ALLOW` до рівнів 1–3 і `DENY (Insufficient clearance)` до `cloud_configs`, `infrastructure_code`, `secrets_vault`.
- `qa_tester` і `partner_access` (допуск 2) отримують `ALLOW` до ресурсів рівнів 1 і 2. Це `shared_resources`, `public_endpoints`, `test_environments`, `partner_apis`, `build_artifacts`. Ресурси рівнів 3 і 4 для них закриті.
- `migrated_user` неактивний і одночасно заблокований. Блокування перевіряється раніше, тому для всіх ресурсів причина `User is blocked`.

`container_breach` і `pipeline_compromise` є в блокуванні, але їх немає в словнику користувачів. Програма перевіряє їх окремо, результат — `DENY (User not found)`.

Єдиний неактивний користувач варіанта вже заблокований, тому гілка `Account inactive` на цих даних не спрацьовує. Після основної перевірки програма додає запис `inactive_account`: його немає в `blocked_users`, а `active` дорівнює `False`. Для нього причина відмови — `Account inactive`. Словник користувачів із методички не змінюється.

Завдання 3. Хешування, CSV і журнал

Для варіанта 9 алгоритм — SHA-224, мінімальна довжина пароля — 13 символів. Сіль береться з номера варіанта і доповнюється нулями зліва до п'яти знаків: 00009.

`generate_hash(password, salt="00000")` повертає шістнадцятковий хеш рядка «пароль + сіль». Якщо пароль або сіль порожні (`None` або `" "`), функція піднімає `ValueError`. Якщо пароль коротший за 13 символів, піднімається `ValidationError`.

Реєструються десять записів:

Логін	Пароль
cloud_architect	CloudVault@Secure13
devops_engineer	Pipeline\$Deploy14
qa_tester	Quality@Assure15
partner_access	Partner#Access16
infra_admin	Infra!Admin2024X
secrets_manager	Secrets@Vault17
build_operator	Build\$Operator18
audit_reader	Audit@Reader19
api_gateway	Gateway#Secure20
backup_agent	Backup@Agent2024Z

`create_users` створює каталог `labs/lab01/data`, якщо його немає, і записує `users.csv` у форматі логін, хеш. Заголовок у файл не пишеться. Повторний запуск перезаписує файл. Після запису програма читає CSV назад і виводить таблицю логін — хеш.

`login` повертає `True`, якщо логін є в базі і хеш уведеного пароля з сіллю 00009 збігається зі збереженням. Невідомий логін і неправильний пароль дають `False`. Порожній логін або пароль дають `ValueError`.

Декоратор `log_event` стоїть на `login` і дописує кожну спробу в `labs/lab01/data/log.json`. Подія містить поля `event`, `user`,

`result` (success або failure), `timestamp` у форматі `PPPP-MM-DD TT:XX:CC`, `args` і `kwargs`. `ValueError` і `ValidationError` теж пишуться в журнал як `failure`, після чого виняток іде далі.

Сценарій показує п'ять спроб:

- правильні дані `cloud_architect` — `True`;
- той самий логін і чужий пароль — `False`;
- невідомий логін — `False`;
- порожній логін — `ValueError`;
- пароль `Short#Pass1` (11 символів) — `ValidationError`.

Робота з файлами зібрана в одному `try`. Окремо перехоплюються `FileNotFoundError`, `PermissionError`, `IOError`, `ValidationError` і `ValueError`. `FileNotFoundError` і `PermissionError` стоять вище за `IOError`, бо в Python 3 вони є його підкласами.

Перевірка коду

Код оформлений за PEP-8: відступ 4 пробіли, довжина рядка не більша за 79 символів, `snake_case` для функцій і змінних, `UPPER_CASE` для констант. У `ruff.toml` увімкнена перевірка довжини рядка. Імporti в `main.py` стоять після `sys.path.append`, як у прикладі методички.

Перевірка в активованому середовищі:

```
ruff check .
ruff format --check .
```

Обидві команди проходять без зауважень.

Висновки

Зроблено три модулі варіанта 9 і спільний запуск у `main.py`.
Аналізатор додає три випадкові копії паролів і виставляє один із п'яти рівнів. Контроль доступу замінює число рівня назвою і перевіряє доступ у порядку з методички. Реєстрація зберігає SHA-224 пароля з сіллю `00009`, звіряє хеш під час входу і пише спроби в `log.json`. Дані студента лежать в `shared/student.py`.
Службовий каталог середовища, база і журнал у репозиторій не входять.